Trabajo Práctico — Parte 2: Floorplan y placement con LibreLane (sky130)

En la parte 1 sintetizaste el CPU Ibex con las macros de SRAM de su caché de instrucciones. En esta parte lo llevás del netlist a un layout con todas las celdas ubicadas: **floorplan** (tamaño del chip, macros, alimentación) y **placement** (ubicación de las celdas estándar). Las etapas restantes (árbol de clock, routing y signoff) quedan para la parte 3.

Se trabaja **solo con el Ibex y la biblioteca `hd`**, partiendo del netlist del TP1: acá no se sintetiza.

Objetivos

Al terminar este trabajo práctico deberías poder:

1. Definir el **footprint** de un bloque (área y pines) y dimensionar su **floorplan**: die, core, filas de celdas estándar y utilización.
2. Analizar la **ubicación de macros**: sus pines, su conectividad con la lógica y el espacio que dejan para las celdas y los cables.
3. Explicar para qué sirven las **celdas físicas** (tap, endcap) y cómo se arma la **red de alimentación** (PDN).
4. Interpretar los resultados de un **placement**: densidad, congestión, largo de cables (HPWL) y la reparación eléctrica del diseño.
5. Comparar el **timing** antes y después del placement y explicar las diferencias.
6. Usar la **GUI de OpenROAD** para inspeccionar y medir un layout.

Antes de empezar

- El repositorio es https://github.com/miguel9554/up-tp2. Se usan el mismo contenedor y el mismo PDK del TP1: no hay que instalar nada nuevo.
- Leé **todo el `README.md`**. Ahí está cómo importar el netlist del TP1, qué hace cada paso del flujo, qué contiene cada reporte y cómo se usa la GUI. Este TP supone que ya lo leíste.
- Necesitás la corrida `hd` del Ibex del TP1 **con la configuración original** (100 ns). Si no la tenés, corré `make sta DESIGN=ibex` en el proyecto del TP1.
- Todos los comandos se corren desde la raíz del proyecto. La GUI tiene que abrirse desde una sesión gráfica de Ubuntu (sección 1.2 del `README.md`).

Cosas a tener en cuenta durante todo el TP

- **Cada corrida crea un directorio nuevo** en `designs/ibex/runs/`. Usá `NAME=` para identificar cada experimento (`make place NAME=centro`) y anotá qué corrida corresponde a cada uno.

- `runs/compare.md` se **sobrescribe** cada vez que corrés `make compare`. Si lo vas a usar en el informe, copialo con otro nombre.
- Cuando un ejercicio te pida modificar `config.json` o `macro_placement.cfg`, **volvé a la versión original** antes de pasar al siguiente. Usá `git diff` para controlarlo: la única diferencia que tiene que quedar es el `DIE_AREA` que escribe `make footprint`.
- El timing es todavía **pre-CTS**: el clock es ideal. Y los cables son una **estimación** a partir del placement, no los cables ruteados.
- Las imágenes de la GUI se guardan con `save_image <archivo>.png` en su consola Tcl (sección 5 del `README.md`).

Entrega

Un **informe en PDF** que contenga:

- Las **tablas** pedidas, con los valores obtenidos y el nombre de la corrida de la que salen.
- Las **respuestas** a cada pregunta, justificadas con datos de los reportes o de la GUI (citá el archivo y el valor). Una respuesta sin justificación con datos no se considera completa.
- Las **capturas de la GUI** pedidas, con una breve descripción de qué se ve en cada una.
- El comando con el que generaste tu footprint (`AREA` y `ASPECT`).

Las preguntas están numeradas (**P1**, **P2**, ...): usá esa numeración en el informe.

Uso de herramientas de IA y defensa del trabajo

- **Está permitido usar chats o agentes de IA** (ChatGPT, Claude, Copilot, etc.) para hacer este trabajo: para entender conceptos, interpretar reportes, escribir comandos o revisar el informe.
- En la **evaluación final el trabajo deberá ser defendido** oralmente. Vas a tener que explicar cada respuesta, cada número y cada decisión de tu floorplan, de dónde sale y qué significa, sin ayuda. Usá la IA para aprender, no para entregar algo que no entendés.

Ejercicio 0: Footprint

El **footprint** de un bloque es el rectángulo que va a ocupar en el chip y la ubicación de sus pines. No lo decide quien hace el PnR del bloque: viene dado de antes (por quien arma el chip completo, que tiene que encajar todos los bloques y conectarlos entre sí). El floorplan **parte** del footprint y del netlist sintetizado.

Acá el footprint lo generás vos con un script, a partir de un área y una relación de aspecto. Los pines se reparten uniformemente sobre los cuatro bordes, en el orden en que están declarados en el netlist (sección 2.2 del `README.md`).

Qué hacer:

1. Importá el netlist de una corrida `hd` del TP1:

   ```
   make import-tp1 TP1_RUN=../up-tp1/designs/ibex/runs/<corrida>
   ```

2. Calculá el área del die (P1) y generá el footprint:

   ```
   make footprint AREA=<área del die en µm²> ASPECT=1
   ```

3. Abrilo en la GUI (P2):

   ```
   make gui-footprint
   ```

Qué se debe lograr: un footprint dimensionado con criterio, y entender qué contiene.

- **P1.** ¿Qué corrida del TP1 importaste? Usando el área de celdas estándar y de macros del TP1 (`designs/ibex/tp1/reports/area.md`), calculá el área de die necesaria para que las instancias ocupen alrededor del **50 %** del core, con un aspect ratio de 1 (tené en cuenta que entre el die y el core hay un margen de unos 10 µm). Generá el footprint con ese valor. ¿Qué dimensiones tiene el die? (La ubicación de las macros que trae el repositorio necesita un die de al menos ~1625 µm de lado.) Más adelante (P7) vas a comparar la utilización que esperabas con la que resultó.
- **P2.** Abrí el footprint en la GUI y relacioná lo que ves con el archivo `designs/ibex/footprint.def`. Incluí una captura del footprint completo.
  - Elegí dos pines del DEF, uno de un borde horizontal (por ejemplo `clk_i`) y uno de un borde vertical. Ubicalos en la GUI (podés acercarte con `gui::zoom_to x0 y0 x1 y1`, en µm, en la consola Tcl) y hacé clic en cada uno. Compará el rectángulo (`BBox`) que muestra el Inspector con las líneas `LAYER` y `PLACED` del DEF: ¿cómo se obtiene uno a partir de las otras? ¿En qué unidades está el DEF?
  - Apagá y prendé `met2` y `met3` en `Layers`: ¿qué bordes tienen pines en cada capa? ¿Por qué tiene sentido esa elección (pensá en la dirección preferida de cada capa)?
  - Medí con la regla (`K`) la separación entre dos pines vecinos
  - ¿En qué bordes quedaron la interfaz de instrucciones (`instr_*`) y la de datos (`data_*`)?
  - ¿Qué variable de `config.json` le indica a LibreLane que use este archivo?

Ejercicio 1: Floorplan

1.1 Correr el floorplan

Qué hacer:

1. Leé `designs/ibex/config.json`, `constraints.sdc` y `macro_placement.cfg`. Este último ya trae la ubicación de las 6 macros de SRAM: no hay que cambiarla.
2. Corré el floorplan y abrilo en la GUI:

```
make floorplan
make gui
```

Qué se debe lograr: entender qué datos necesita el floorplan y por qué las macros se ubican antes que todo lo demás.

- **P3.** `constraints.sdc` agrega `unset_propagated_clock [all_clocks]` respecto del TP1. Explicá la diferencia entre un clock **ideal** y uno **propagado**. ¿Por qué en esta etapa conviene el ideal? Para justificarlo, buscá en el netlist (`designs/ibex/tp1/ibex_top.nl.v`) la celda de clock gating (`dlclkp`): ¿cuántos flip-flops maneja su salida? ¿Qué slew tendría ese net sin árbol de clock?
- **P4.** ¿Por qué la red de alimentación necesita que las macros ya estén fijas? ¿Y por qué, en general, las macros se ubican **antes** que las celdas estándar?

1.2 Las macros de SRAM

Cada línea de `macro_placement.cfg` tiene el nombre de la instancia, `x` e `y` (esquina inferior izquierda de la macro, en µm, respecto de la esquina inferior izquierda del die) y la orientación. Las orientaciones posibles (las de DEF) son:

Orientación	Transformación respecto de la macro original
`N`	ninguna (como está dibujada en el LEF)
`S`	rotada 180°
`FN`	espejada respecto del eje vertical (izquierda ↔ derecha)
`FS`	espejada respecto del eje horizontal (arriba ↔ abajo)

La coordenada `x y` es siempre la esquina inferior izquierda del rectángulo que ocupa la macro **ya orientada**.

Qué se debe lograr: entender por qué las macros están donde están.

- **P5.** La macro es `sky130_sram_1kbyte_1rw1r_32x256_8` y su LEF está en `$PDK/libs.ref/sky130_sram_macros/lef/` (sección 4 del `README.md` del TP1).
  - ¿Cuáles son sus dimensiones? ¿Cuántas macros entran a lo ancho del core que calculaste en P1?
  - La macro tiene dos puertos: el 0 (lectura/escritura) y el 1 (solo lectura). Mirando `designs/ibex/src/prim_ram_1p.sv` **del TP1**, ¿cuál de los dos se usa? ¿Qué se hace con el otro?

  - ¿En qué **bordes** de la macro están los pines de datos, dirección y control del puerto usado (`din0`, `dout0`, `addr0`, `csb0`, `web0`, `wmask0`, `clk0`)? Sacalo del LEF (las coordenadas `RECT` de cada `PIN`) y verificalo en la GUI: hacé zoom sobre una macro y seleccioná algunos pines.
  - ¿Y los pines de alimentación (`vccd1`, `vssd1`)? ¿En qué capa de metal están?
- **P6.** Analizá la ubicación de las macros. Incluí una captura de la GUI con las 6 macros.
  - ¿Cuánto mide el **canal** entre macros vecinas, y entre las macros y el borde del core? ¿Para qué hace falta dejar ese espacio?
  - La fila de abajo está en `S` y la de arriba en `N`. ¿Hacia dónde quedan mirando los pines del puerto usado en cada fila? ¿Por qué tiene sentido?
  - Mirando los pines del footprint (P2), ¿qué macro queda sobre la interfaz de instrucciones? ¿Qué consecuencia esperás que tenga?

1.3 Analizar el floorplan

Qué hacer: con el floorplan que funciona, leé `<run>/reports/floorplan.md` y abrí la corrida en la GUI.

Qué se debe lograr: entender la estructura del core sobre la que se van a ubicar las celdas.

- **P7.** Completá esta tabla y compará la utilización con la que esperabas en P1:

Ítem	Valor
Die (ancho × alto, µm)	
Core (ancho × alto, µm)	
Margen entre die y core (µm)	
Cantidad de filas	
Sites por fila	
Utilización efectiva	

  Buscá las dimensiones del site `unithd` (`SITE unithd` en `$PDK/libs.ref/sky130_fd_sc_hd/techlef/sky1` y verificá que filas × alto del site = alto del core, y sites × ancho del site = ancho del core. ¿Qué relación hay entre la altura del site y la altura de las celdas que viste en el TP1 (P17)?

- **P8.** Celdas físicas. En la GUI (`STEP=08`), activá las filas (`Rows`) y buscá las celdas endcap y tap (son instancias: seleccioná alguna y mirá su celda en el Inspector).
  - ¿Qué función cumple cada tipo? ¿Por qué hacen falta en una tecnología CMOS de pozo (*well*)?

  - ¿Cuántas hay de cada una y qué área ocupan? Compará el área de las tap con el área de celdas estándar del diseño.
  - Medí con la regla la distancia entre dos tap consecutivas de una misma fila. ¿De qué regla de diseño crees que sale?
  - ¿Qué pasó con las filas en la zona de las macros (paso 7)?

- **P9.** Red de alimentación. En la GUI (`STEP=10`), mirando una capa por vez en `Layers`:
  - Identificá los *rails* de las filas, los *straps* verticales y los horizontales. ¿En qué capa de metal está cada uno? ¿Cada cuántos µm se repiten los straps?
  - ¿Pasan straps por encima de las macros? ¿En qué capa y por qué en esa?
  - ¿Por dónde **entran VPWR y VGND** al bloque? Buscá los pines `VPWR` y `VGND` en la sección `PINS` del DEF del paso 10 (`<corrida>/10-openroad-generatepdn/ibex_top.def`), y en la GUI hacé clic sobre un strap. ¿Qué forma y en qué capas están esos pines? ¿Por qué no hay un pin de alimentación en el borde, como los de señal, ni aparecen en `footprint.def`? ¿Cómo se conectaría el chip completo a la alimentación de este bloque?
  - Seguí el camino de VPWR desde el pin del bloque hasta el pin `VPWR` de una celda estándar: ¿qué capas y qué vías atraviesa?
  - Los pines de alimentación de la SRAM se llaman `vccd1`/`vssd1`, y los de las celdas estándar `VPWR`/`VGND`. ¿Qué hace la variable `PDN_MACRO_CONNECTIONS` de `config.json`? Hacé una prueba: borrala, corré `make floorplan NAME=sin_pdn_macros`, y buscá los warnings `PDN-0231` en la salida. ¿Qué habría pasado con las SRAM en el chip? Después **volvé a agregarla**.

Ejercicio 2: Placement

2.1 Correr el placement

Qué hacer:

```
make place NAME=ref
```

Tarda 2-3 minutos. Leé `qor.md`, `placement.md` y `timing.md` de la corrida y abrila en la GUI.

Qué se debe lograr: saber leer el resultado de un placement y relacionar cada número con lo que se ve en el layout.

- **P10.** Completá esta tabla:

Métrica	Valor
Área de macros (µm²)	
Área de celdas estándar (µm²)	
Utilización (todas las instancias)	
Utilización de celdas estándar	
Tap cells: cantidad y área (µm²)	
Buffers de reparación insertados	
Wirelength estimado, HPWL (µm)	
Congestión después del placement global	

¿Por qué la utilización total y la de celdas estándar son tan distintas? ¿Qué dice eso sobre el tamaño de die que elegiste en el footprint? ¿La cantidad de celdas estándar cambió respecto del netlist del TP1? ¿Por qué?

- **P11.** Ubicación de las celdas.
  - Incluí una captura del placement completo y otra del mapa de calor `Placement Density`. ¿Las celdas se reparten uniformemente en el área libre? ¿Dónde se concentran?
  - Los **pines de entrada/salida** vienen del footprint (paso 2). ¿Cómo influyen, junto con los pines de las macros, en dónde se agruparon las celdas? Elegí un bus (por ejemplo `instr_addr_o`) y mirá dónde quedaron las celdas conectadas a sus pines.
  - Ubicá dónde quedaron al menos tres módulos del CPU: el banco de registros, la etapa de fetch (que incluye la icache) y el bloque de ejecución (que incluye la ALU). ¿Tiene sentido su ubicación respecto de las macros y de los pines?

    El **Hierarchy Browser** de la GUI no sirve para esto: aparece plano. La jerarquía se perdió en la **síntesis del TP1**: LibreLane sintetiza por defecto con `SYNTH_HIERARCHY_MODE = flatten`, y Yosys aplana todo el diseño en un solo módulo (`ibex_top`) y renombra casi todas las celdas (`_25062_`, ...). Solo las 6 SRAM y el clock gate conservan su nombre jerárquico. Los que sí conservan el nombre del módulo son muchos **nets**, por ejemplo `gen_regfile_ff.register_file_i.g_plain_rf.g_rf_flops[10].rf_reg_q[0]`.

    Para ver dónde quedó un módulo, seleccioná y resaltá sus nets desde la consola Tcl de la GUI. `-name` es una expresión con comodines, y cada número de `-highlight` (de 0 a 7) es un grupo con un color distinto:

    ```
    # banco de registros
    select -type Net -name "*register_file_i*" -highlight 0
    # etapa de fetch, con la icache
    ```

```
select -type Net -name "*if_stage_i*" -highlight 1
# bloque de ejecución: ALU y multiplicador
select -type Net -name "*ex_block_i*" -highlight 2
```

Cada comando imprime cuántos nets seleccionó. El Inspector permite recorrerlos uno por uno (`Next`/`Previous`). Otros prefijos útiles: `cs_registers_i` (CSRs), `id_stage_i` (decodificación) y `load_store_unit_i`. Para borrar el resaltado y la selección:

```
# borra el resaltado del grupo 0
gui::clear_highlights 0
# borra el resaltado de todos los grupos
gui::clear_highlights -1
# vacía la selección (el Inspector)
gui::clear_selections
```

Tené en cuenta que así ves los cables con nombre del módulo (en general, sus registros y señales de interfaz), no todas sus celdas: es una ubicación aproximada.

- **P12.** Placement global y detallado. Abrí la misma corrida con `STEP=13` y con `STEP=19` y hacé zoom sobre una misma zona chica.
  - ¿Qué diferencia ves entre las celdas después del placement global y después del detallado? ¿Qué condiciones cumple un placement *legal*?
  - Según `placement.md`, ¿cuánto se infló el área de celdas en el placement global para reducir la congestión? ¿Qué significa ese "inflado"?

2.2 Reparación y timing

Qué se debe lograr: relacionar la reparación eléctrica y los cables estimados con lo que viste en el TP1.

- **P13.** Reparación del diseño (`placement.md`, sección *Design repair*).
  - ¿Qué insertó o modificó la herramienta? Explicá para qué sirve cada tipo: buffers de entrada, de salida, de reparación, celdas *tie*, cambios de tamaño.
  - Compará las violaciones de max slew y max cap del TP1 con las de después del placement (`timing.md`, misma esquina `tt`). Relacionalo con tu respuesta a la P15 del TP1: ¿por qué recién ahora se pueden reparar?
  - Quedan algunas violaciones de max slew. ¿En qué pines están? (Buscalas en `<run>/20-openroad-stamidpnr-1/checks.rpt`.) Buscá el límite `max_transition` de esos pines en el `.lib` de la SRAM y compáralo con los slews típicos de tu diseño. ¿Te parece un límite razonable? ¿Qué harías al respecto en un diseño real?
  - Buscá en la GUI los nets `rst_ni` y `clk` (botón **Find**) y mirá su fanout en el Inspector. En el TP1 ambos manejaban miles de pines. ¿Cuál

de los dos fue bufferizado y cuál no? ¿Por qué la herramienta deja uno sin tocar, y qué etapa se va a encargar de él?

- **P14.** Timing (`timing.md`). Completá:

Esquina `tt`	TP1 (pre-PnR)	Después del placement
Setup WS (ns)		
Hold WS (ns)		
Violaciones de max slew		
Violaciones de max cap		

  - Ahora los cables tienen resistencia y capacidad, pero el slack de setup **¿empeoró o mejoró?** Explicalo comparando el camino crítico de los dos reportes: ¿empieza y termina en el mismo lugar? ¿Qué celdas o nets que antes eran lentos ya no aparecen, y por qué? (Recordá lo que viste sobre fanout y slew en la P14 del TP1.)
  - En el camino crítico de ahora, elegí dos nets: uno corto y uno que una celdas lejanas. ¿Cuánto retardo agrega cada uno (la diferencia de tiempo entre el pin de salida que lo maneja y el pin de entrada siguiente)?

- **P15.** En la GUI, pestaña **Timing Report** → `Update`, seleccioná el peor camino de setup. Incluí una captura con el camino dibujado sobre el layout. ¿Qué recorrido hace en el chip? Medí con la regla la distancia entre su inicio y su fin. ¿Hay alguna forma de acortarlo cambiando el floorplan?

Ejercicio 3 (opcional): Experimentos de floorplan

Este ejercicio es **opcional**: no hace falta para aprobar el TP. Si lo hacés, incluilo en el informe con el mismo nivel de detalle que los anteriores.

Cada experimento cambia **una sola cosa** respecto de tu corrida de referencia (`NAME=ref`); el footprint es siempre el mismo. Corré cada uno con un `NAME` distinto y, al final, armá la tabla comparativa con `make compare`. Para cada experimento, incluí una captura de la GUI y explicá **cada diferencia** respecto de la referencia (wirelength, congestión, buffers, violaciones, slack), no solo si empeoró o mejoró.

Con un clock de 100 ns sobra mucho tiempo, así que el slack va a cambiar poco entre experimentos: las diferencias importantes están en el **wirelength**, la **congestión** y la **reparación** (buffers, violaciones de max cap). Una diferencia de wirelength de algunos puntos porcentuales es significativa: se traduce en área de cables, potencia y, a frecuencias más altas, en timing.

- **P16. Orientación de las macros.** Cambiá la orientación de una fila de macros para que los pines del puerto usado miren **hacia el borde del**

die en vez de hacia las celdas (`NAME=pines_afuera`).

- **P17. Ubicación de las macros.** Ubicá las macros en el **centro** del core, en dos filas, de modo que las celdas estándar queden repartidas arriba y abajo de ellas (`NAME=centro`). ¿Qué pasa con los caminos que van de las SRAM a los pines del chip y a la lógica del CPU?
- **P18. Densidad objetivo.** Agregá `"PL_TARGET_DENSITY_PCT"` a `config.json` con valores 30 y 90 (`NAME=dens30`, `NAME=dens90`). ¿Qué valor usó la herramienta en la referencia (`placement.md`)? Compará los mapas de densidad y de congestión. ¿Qué compromiso hay entre una densidad alta y una baja?

Criterios de evaluación

Se valora especialmente que las decisiones de floorplan estén **justificadas** y que las respuestas se apoyen en **números de los reportes y mediciones en la GUI**, relacionándolos con los conceptos de la materia: conectividad y largo de cables, congestión, alimentación, fanout y slew, setup/hold.